Q12. K-Means Clustering

Original Aim: Use the four numerical attributes of the Iris dataset to group flowers into three clusters without using Species during clustering. Apply K-Means, plot the clusters using two attributes report the cluster assignments.

Aim

Group flowers into three clusters using all four numerical features without Species, report assignments and plot a two-attribute view.

Requirements

Python, Pandas, Matplotlib, StandardScaler and KMeans. Reference: [D13].

Essential theory

K-Means is unsupervised: it receives measurements but no correct species labels. It alternates between assigning rows to the nearest centroid and updating centroids as cluster means, seeking low within-cluster squared distance (inertia). A cluster is a group of similar feature vectors under that distance rule. Scaling matters because larger-spread features would otherwise dominate Euclidean distances.

This is descriptive clustering of the whole dataset, not a supervised train/test prediction assessment. Choosing k=3 follows the assigned question; it is not evidence that exactly three clusters is optimal by an independent model-selection criterion. Cluster IDs 0, 1 and 2 are arbitrary names, not species codes.

Step-by-step procedure

1. Select only df[features]; Species is not passed to the scaler or KMeans.
2. Standardize the four features for equal-scale distance calculations.
3. Fit KMeans with three clusters, k-means++ initialization, ten initializations and fixed seed 42.
4. Attach fit_predict's cluster assignments to a feature-only copy, retaining row order.
5. Print all assignments and save them to q12_assignments.csv with original row index.
6. Convert centroids back to centimetres for interpretation and plot petal length/width by cluster. The fit still uses all four features.

Complete Python program

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
features = [
    "SepalLengthCm", "SepalWidthCm",
    "PetalLengthCm", "PetalWidthCm"
]
species = ["Iris-" + name for name in iris.target_names]
df = pd.DataFrame(iris.data, columns=features)
df["Species"] = pd.Categorical.from_codes(
    iris.target, categories=species
)

X = df[features]  # Species is not an input to scaling or clustering.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = KMeans(n_clusters=3, init="k-means++", n_init=10,
               random_state=42, algorithm="lloyd")
clusters = model.fit_predict(X_scaled)
assignments = X.copy()
assignments["Cluster"] = clusters
assignments.to_csv("q12_assignments.csv", index_label="Row")
centroids_cm = pd.DataFrame(
    scaler.inverse_transform(model.cluster_centers_), columns=features
)
centroids_cm.index.name = "Cluster"
print("ALL CLUSTER ASSIGNMENTS (original row index)")
print(assignments.to_string())
print("CLUSTER COUNTS")
print(assignments["Cluster"].value_counts().sort_index().to_string())
print("CENTROIDS IN ORIGINAL CM UNITS")
print(centroids_cm.round(4).to_string())
print(f"Inertia in standardized space: {model.inertia_:.6f}")
fig, ax = plt.subplots(figsize=(7, 4.5))
colors = ["#147d92", "#dc8b26", "#7357a6"]
for cluster, color in enumerate(colors):
    mask = clusters == cluster
    ax.scatter(X.loc[mask, "PetalLengthCm"],
               X.loc[mask, "PetalWidthCm"],
               label=f"Cluster {cluster}", color=color, s=30, alpha=0.8)
ax.scatter(centroids_cm["PetalLengthCm"], centroids_cm["PetalWidthCm"],
           marker="X", s=150, color="black", label="Projected centroids")
ax.set_xlabel("PetalLengthCm (cm)")
ax.set_ylabel("PetalWidthCm (cm)")
ax.set_title("K-Means: 4-feature clustering shown in 2 dimensions")
ax.legend(fontsize=8)
fig.tight_layout()
fig.savefig("q12_clusters.png", dpi=180)
plt.show()
```

Code explanation

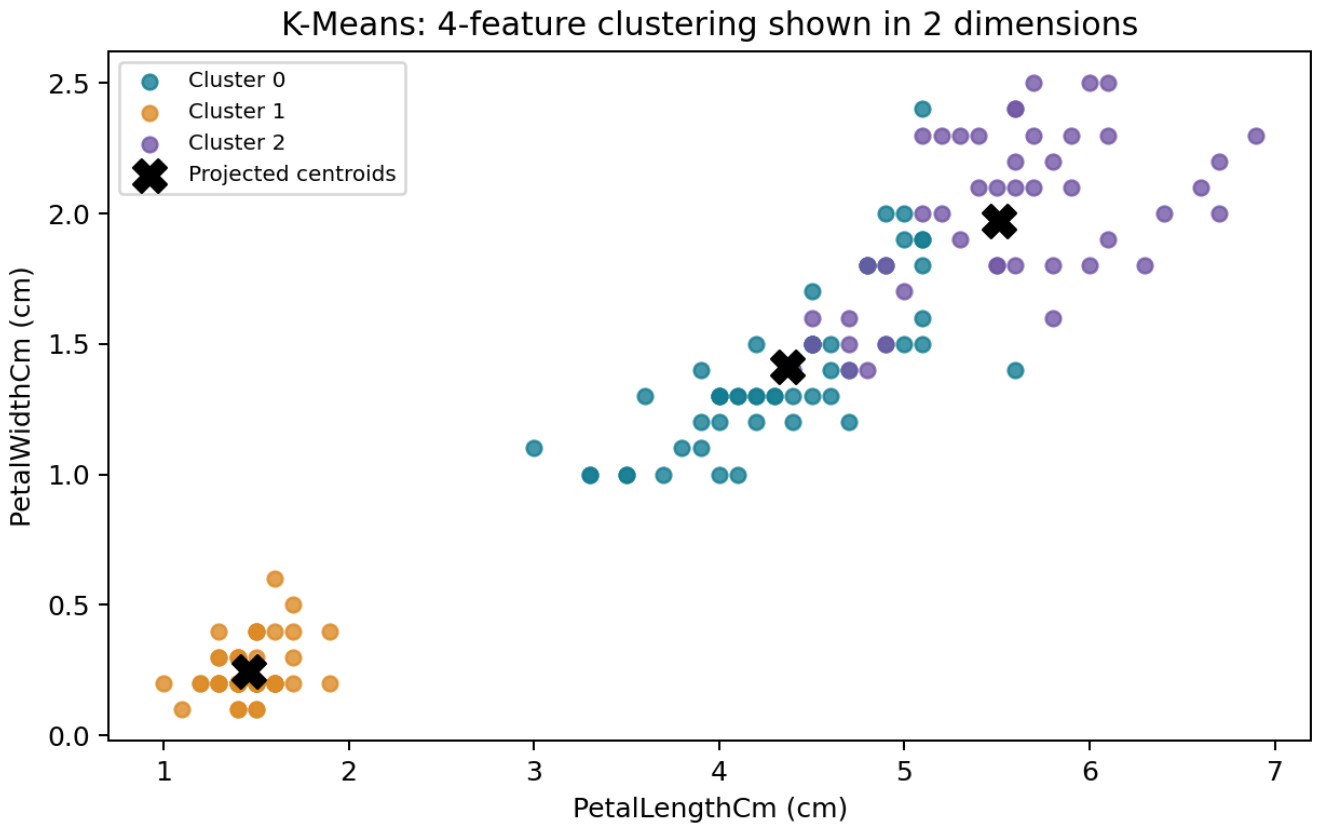
- `n_init=10` explicitly tries ten centroid initializations and retains the best run by inertia, avoiding dependence on version-changing defaults.
- `random_state=42` fixes random choices for this software/data setup; label numbering or small numeric differences can still vary across versions.
- `fit_predict` both learns centroids and returns one cluster ID for every row. It never sees Species.
- `inverse_transform` returns standardized centroids to the original measurement units. They are means, not necessarily existing flowers.

- The scatter uses original centimetre coordinates. Black X markers show centroid projections onto those two attributes; distances were optimized in all four standardized dimensions. Different clusters can overlap in this 2D view.

Verified output summary

```
CLUSTER COUNTS
Cluster
0    53
1    50
2    47
CENTROIDS IN ORIGINAL CM UNITS
         SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
Cluster
0               5.8019        2.6736         4.3698        1.4132
1               5.0060        3.4280         1.4620        0.2460
2               6.7809        3.0957         5.5106        1.9723
Inertia in standardized space: 139.820496
```

Verified cluster figure



Complete row-to-cluster assignments

The following compact table contains all 150 assignments from the saved q12_assignments.csv. Row means the original zero-based Iris DataFrame index, not an extra input feature. Three side-by-side row/cluster pairs conserve space; read each pair independently.

Row	Cluster	Row	Cluster	Row	Cluster
0	1	50	2	100	2
1	1	51	2	101	0
2	1	52	2	102	2
3	1	53	0	103	2
4	1	54	0	104	2
5	1	55	0	105	2
6	1	56	2	106	0
7	1	57	0	107	2
8	1	58	0	108	2
9	1	59	0	109	2
10	1	60	0	110	2
11	1	61	0	111	2
12	1	62	0	112	2
13	1	63	0	113	0
14	1	64	0	114	0
15	1	65	2	115	2
16	1	66	0	116	2
17	1	67	0	117	2
18	1	68	0	118	2
19	1	69	0	119	0
20	1	70	2	120	2
21	1	71	0	121	0
22	1	72	0	122	2
23	1	73	0	123	0
24	1	74	0	124	2
25	1	75	2	125	2
26	1	76	2	126	0
27	1	77	2	127	2
28	1	78	0	128	2
29	1	79	0	129	2
30	1	80	0	130	2
31	1	81	0	131	2
32	1	82	0	132	2
33	1	83	0	133	0
34	1	84	0	134	0
35	1	85	2	135	2
36	1	86	2	136	2
37	1	87	0	137	2
38	1	88	0	138	0
39	1	89	0	139	2
40	1	90	0	140	2
41	1	91	0	141	2
42	1	92	0	142	0
43	1	93	0	143	2
44	1	94	0	144	2
45	1	95	0	145	2
46	1	96	0	146	0
47	1	97	0	147	2
48	1	98	0	148	2
49	1	99	0	149	0

Observation

For this run, clusters 0, 1 and 2 contain 53, 50 and 47 flowers. Cluster 1 has the smallest mean petals, while cluster 2 has the largest mean petals; the printed centroids show those differences numerically. The plot shows a compact short-petal group and overlap among some larger-petal points. These are inferred clusters, not automatically correct species classifications. No classification accuracy is reported because labels were not used for clustering or matched to arbitrary cluster IDs.

Result

All 150 flowers were clustered reproducibly from four standardized measurements, every row received an assignment, and the two-attribute cluster plot was saved with a legend and axis units.

Common errors and troubleshooting

- Species accidentally included in X: use the explicit features list, not the whole DataFrame or target-derived indicators.
- Different cluster numbers in another run: IDs can permute; compare groups/centroids, data, seed and parameters rather than interpreting IDs as species codes.
- One feature dominates: standardize before Euclidean-distance clustering.
- CSV row count looks like 151: the header is not an observation; the data contain 150 rows.

Viva questions and answers

- Why is K-Means unsupervised? It learns groups without target labels.
- What is a centroid? The mean feature vector of a cluster in the space being fitted.
- Is cluster 0 the same as species code 0? No; cluster IDs have arbitrary ordering.
- Why can clusters overlap on this plot? The model used four dimensions; the plot shows only two.